Layer-Interaction Adaptive Pruning for Remote Sensing Scene Classification

Yiheng Lu, Zhuping Hu, Wei Zhao, *Senior Member, IEEE*, Ziyu Guan, *Senior Member, IEEE*, Maoguo Gong, *Fellow, IEEE*, and Mingyang Zhang, *Member, IEEE*

***Abstract*— The advancement of convolutional neural networks (CNNs) has enhanced remote sensing scene classification on satellites. However, the increased computational complexity of CNNs will impose a substantial burden on satellite hardware, thereby hindering the practical application. Although various pruning techniques have been developed to reduce the scale of CNNs by assessing the importance of model parameters, these weight-based methods are often accompanied by a degradation in model performance when the original model is undertrained and fails to provide meaningful parameters. In this article, we introduce a novel layer-interaction adaptive pruning (LiAP) method designed to streamline undertrained models. Unlike conventional approaches, LiAP evaluates the importance of neurons based on the distribution of eigenvalues rather than individual parameters. Specifically, this is achieved by projecting the weight matrix of each convolutional layer into the eigenspace, where the eigenvalues are utilized to assess the redundancy of filters. Then each space will be assigned a fined-score based on the eigenvalue distribution to provide a robust measure of filter importance. Compared to weight-based pruning methods, LiAP maintains consistent evaluation accuracy between well-trained and undertrained models, as the eigenspace is inherently robust to variations in the weight parameter space. We conducted extensive experiments on VGG-16 and ResNet-50 architectures using datasets such as AID, NWPU-RESISC45, PatternNet, and WHU-RS19 over various data partitions. Notably, our method achieved state-of-the-art (SOTA) reductions in both FLOPs and parameters for VGG-16 across the aforementioned datasets. These results underscore the efficacy of LiAP in enhancing the efficiency and applicability of CNNs in satellite-based remote sensing tasks.**

***Index Terms*— Convolutional neural network (CNN), eigenvalue-based space scoring, remote sensing.**

Received 18 February 2025; revised 21 April 2025; accepted 12 May 2025. Date of publication 16 May 2025; date of current version 29 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62425605, Grant 62133012, Grant 62303366, and Grant 61936006; in part by the Key Research and Development Program of Shaanxi under Grant 2025CY-YBXM-042, Grant 2025CY-YBXM-041, and Grant 2022ZDLGY01-10; and in part by the Fundamental Research Funds for the Central Universities under Grant ZYTS25090 and Grant ZYTS25086. *(Corresponding authors: Maoguo Gong; Ziyu Guan.)*

Yiheng Lu, Zhuping Hu, Wei Zhao, Ziyu Guan, and Mingyang Zhang are with the Key Laboratory of Collaborative Intelligence Systems, Ministry of Education, Xidian University, Xi'an 710071, China (e-mail: lyhxdu@gmail.com; huzhuping@stu.xidian.edu.cn; ywzhao@mail.xidian.edu.cn; zyguan@xidian.edu.cn; omegazhangmy@gmail.com).

Maoguo Gong is with the School of Electronic Engineering, Key Laboratory of Collaborative Intelligence Systems, Ministry of Education, Xidian University, Xi'an 710071, China, and also with the Academy of Artificial Intelligence, College of Mathematics Science, Inner Mongolia Normal University, Hohhot 010028, China (e-mail: gong@ieee.org).

Digital Object Identifier 10.1109/TGRS.2025.3570653

I. Introduction

THE development of satellite technology has generated a huge amount of remote sensing data to observe Earth's scenarios. In particular, convolutional neural networks (CNNs) are deployed widely on satellites to achieve better remote sensing scene classification via powerful feature identification ability [1], [2], [3], [4], [5], [6], [7], [8]. However, the high-computational costs associated with CNNs, stemming from their large number of stacked neurons (filters), pose significant challenges for satellite hardware when embedding these models. As a result, pruning of CNNs has emerged as a critical approach to address this issue by eliminating redundant filters or parameters. To mitigate computational costs, many works have exploited pruning techniques to remove redundant filters or parameters from original models on remote sensing images [9], [10], [11], [12], [13], [14]. However, these methods rely on well-trained models to provide meaningful weight information, which limits their ability to evaluate filter importance stably. Then, the performance of the undertrained model may degrade after pruning. Further, some neural architecture searching (NAS) methods are proposed to cut filters by model adaptation, that is, customizing a personalized architecture using evolutionary algorithms, reinforcement learning, etc [15], [16], [17], [18], [19]. While NAS methods can achieve higher accuracy for specific datasets, their search phases are computationally intensive and time-consuming, requiring substantial resources. Therefore, a stable pruning technique across undertrained and well-trained models is pressing to achieve model compression on satellites with higher efficiency.

To address the concern of stability in pruning criteria across models with varying degrees of training, we propose an evaluation-efficient and reliable method, layer-interaction adaptive pruning (LiAP) to remove redundant filters in a supervised manner, specifically designed for remote sensing images. LiAP builds on the strengths of EFPF from [20], which evaluates filter importance stably through eigenvalues. However, unlike EFPF, LiAP enables end-to-end model pruning without requiring any hyperparameters, which are critical in EFPF. This eliminates the need for extensive hyperparameter tuning, significantly reducing computational resources and improving efficiency.

In particular, three main steps are involved in LiAP.

1) Using a SVD-based *Eigenvalue Projector* to transfer each weight matrix from parameter space to eigenspace.

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

2) Designing an *adaptive base space selection module* to select a base space of eigenvalues via the variance of eigenvalues.
3) Scoring each eigenspace by the distribution of eigenvalues over the Kullback–Leibler (KL) divergence. Then, each convolutional layer will be assigned a different pruning ratio in terms of the scores of eigenspace, and the removal of filters within each layer is then executed following the random strategies outlined in [20].

The *Eigenvalue Projector* operates by transforming the weight matrix from the parameter space into the eigenspace using singular value decomposition (SVD) [21], [22], [23], [24], [25], a process that incurs minimal information loss. This transformation allows us to utilize the distribution of eigenvalues within the eigenspace to represent the distribution of weight information from the parameter space. Furthermore, the importance of each eigenspace can be assessed based on the shape of its eigenvalue distribution [26]. Specifically, a sharper distribution of eigenvalues indicates lower importance of the corresponding eigenspace, which consequently warrants a higher pruning ratio for the associated convolutional layer. Conversely, a flatter distribution suggests higher importance and a lower pruning ratio. Simultaneously, the distribution of eigenvalues is proven to be robust between undertrained and well-trained models with extensive experiments and mathematical analysis [26], which motivates us to obtain a stable evaluation criterion.

To build a unified score mechanism, we design an *adaptive base space selection module* to achieve the scoring of each eigenspace. This module operates by first identifying the most significant eigenspace, designated as the base space, which is characterized by the flattest distribution of its corresponding eigenvalues. Subsequently, the eigenvalue distributions of all other eigenspaces are compared against this base space to derive their respective importance scores. More precisely, the importance of each eigenspace is primarily determined by the variance of its eigenvalue distribution; the greater the variance, the flatter the distribution, and consequently, the higher the importance attributed to that eigenspace.

During the final scoring phase, the KL divergence is employed to quantify the importance of each eigenspace across whole convolutional layers. Since the shape of the eigenvalue distribution effectively reflects the significance of an eigenspace [26], we compare pairs of distributions to measure their shape differences. These differences are then translated into specific scores using the KL divergence, providing a precise metric for evaluating eigenspace importance.

In conclusion, the LiAP can achieve a stable filter pruning between undertrained and well-trained models by "eigenvalue projection," "selection of base space," and "scoring with KL divergence." To validate its effectiveness, we conduct experiments on VGG-16 [27] and ResNet-50 [28] with AID [29], NWPU-RESISC45 [4], PatternNet [30], and WHU-RS19 [31] with varying data partitions. Remarkably, LiAP consistently outperformed these methods in terms of reducing both FLOPs and parameters, particularly when the original model was undertrained. These results highlight the robustness and efficiency of LiAP in model compression tasks, even under suboptimal training conditions.

There are three contributions to this article.

1) We propose LiAP to achieve a stable importance evaluation between undertrained and well-trained models without hyperparameters. Then LiAP can be easily deployed on edge devices such as satellites.
2) Eigenvalue projector and KL divergence are included in LiAP to quantify the eigenspace of each convolutional layer with fined-score. Consequently, LiAP can remove redundant filters in terms of the eigenspace score efficiently without any hyperparameter.
3) The VGG-16 [27] and ResNet-50 [28] are utilized to verify the efficacy of LiAP on remote sensing datasets AID [29], NWPU-RESISC45 [4], PatternNet [30], and WHU-RS19 [31] with different data partitions. Consequently, LiAP achieved state-of-the-art (SOTA) results.

This article is organized as follows. In Section II, we give an overview of related works about NAS-based and weight-based pruning methods. In Section III, we demonstrate the details of LiAP. In Section IV, various experiments are conducted to verify the efficacy of LiAP on four remote sensing datasets with different data partitions. In Section V, we summarize the article, as well as look forward to the application of LiAP in a wider frontier.

II. Related Work

In this section, we delineate the NAS-based and weight-based pruning techniques employed in model compression, while also scrutinizing the limitations of these methodologies with respect to evaluation efficiency and stability.

A. NAS-Based Pruning Methods

Unlike manually crafted CNNs, NAS techniques are capable of tailoring an optimal architecture for a specific dataset through an iterative search process. Therefore, NAS-based methods can usually yield a relatively small architecture with higher accuracy. Specifically, Zoph and Le [32] and Zoph et al. [33] utilize the recurrent neural network (RNN) controller to build architectures from a well-designed searching space. Real et al. [34] incorporate the evolutionary algorithm into the architectural design of models, engendering a diverse array of candidate subnetworks. Similarly, Jing et al. [35] replace the evolution algorithm with sampling-based searching to obtain a series of subarchitectures by continuous relaxation. Moreover, Peng et al. [36] propose a more efficient bilevel optimization strategy to search for architectures progressively. Besides, other NAS-based methods have been studied broadly in [15], [16], [37], and [38] to search for appropriate structures on remote sensing datasets. However, these approaches are heavily dependent on graphics processing units (GPUs) to conduct iterative architecture searches and candidate evaluation, which impede impractical deployment on edge devices. Consequently, pruning techniques that entail the elimination of filters demonstrate a distinct advantage for model compression on satellite platforms.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

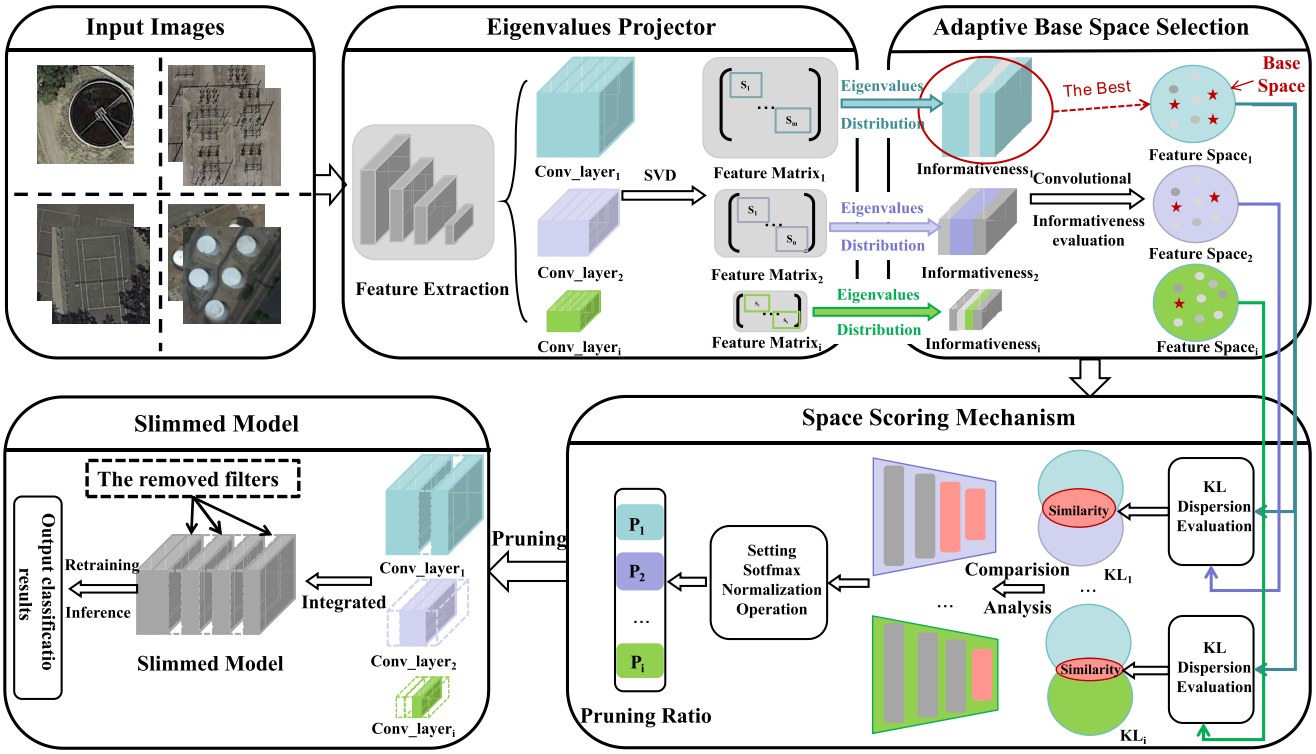


Fig. 1. Overflow of LiAP. The SVD projector will map the weight matrix from parameter space to eigenspace. A base distribution of eigenvalues will be selected in terms of the variance, and then all other distributions will be compared with the base distribution for calculating the KL divergence. Finally, each convolutional layer will be pruned by means of the KL divergence of each eigenspace.

B. Weight-Based Pruning Methods

Pruning techniques can reduce the size of a model by removing parameters or filters without a repeated search process, which can save a significant amount of computational resources. Moreover, pruning can be classified into unstructured and structured types according to the integrity of filters.

Unstructured pruning selectively eliminates a portion of parameters from the original model while preserving the integrity of the filters. Lecun et al. [39] and Hassibi and Stork [40] introduced analogous methodologies termed "optimal brain damage" and "optimal brain surgeon," respectively, aimed at excising superfluous parameters from deep neural networks (DNNs). More recently, Frankle and Carbin [41] explored an innovative metric known as the "Lottery Hypothesis," which identifies redundancies through the masking of initialized parameters. However, unstructured pruning may lead to the sparsity of parameters after pruning, necessitating specialized hardware for efficient computation. Therefore, structured pruning has garnered increased attention in the pruning frontier.

Structured pruning will remove the intact filter to reduce the model size. Then the slimmed model can maintain a compact structure without sparse parameters, which can be applied to normal hardware easily. Li et al. [42] use l_1-norm criterion to evaluate the importance of filters. Molchanov et al. [43] select filters by Taylor expansion at first order. Additionally, Liu et al. [44] and Zhang et al. [45] have proposed distinct approaches for filter removal, with the former assessing the significance of batchnorm neurons and the latter leveraging fusible compensatory convolutions to achieve model simplification. Additionally, several structured pruning methods have been exploited to slim the CNNs [46], [47], [48], [49], [50] by different criteria. However, the above-mentioned pruning methods always depend on weight information, which yields unstable evaluation when the original model is not trained well. Even Lu et al. [20] proposed a structure-aware pruning method energy-based filter pruning framework (EFPF), to achieve stable model evaluation between undertrained and well-trained models. A hard hyperparameter is needed to confirm a well-customized model with extensive trials, which is time-consuming. Thus, we design the LiAP to achieve a stable model evaluation with higher efficiency.

III. Eigenvalue-Based Space Scoring Pruning Framework

In this article, we introduce LiAP, a novel approach designed to provide stable model evaluation between undertrained and well-trained models without the need for hyperparameters. As illustrated in Fig. 1, LiAP consists of three key components.

1) *Eigenvalues Projection:* This component involves the projection of the weight matrix using SVD, transforming weight information from the parameter space into eigenvalues in the eigenspace. Here, S_i represents the ith eigenvalue within a specified eigenspace, and the ith distribution corresponds to the eigenvalue distribution of the ith convolutional layer.
2) *Adaptive Base Space Selection:* This step focuses on selecting the base space through an information evaluation process derived from the corresponding feature matrix. Initially, the variance of eigenvalues within each eigenspace is computed. The eigenspace exhibiting the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

largest variance is then designated as the base space, providing a base distribution that serves as a reference for scoring other eigenspaces.

3) *Space Scoring Mechanism:* This component employs scoring techniques based on KL divergence. Specifically, each eigenvalue distribution is compared with the base distribution to compute the KL divergence. Here, KL_i denotes the KL divergence for the ith eigenspace. All KL divergence values are subsequently processed through a softmax function to calculate a refined score P. Based on the value of P, each convolutional layer is assigned a unique pruning ratio to remove redundant filters.

In Sections III-A–III-C, we will delve into the details of LiAP, structured into: Task Statement, Eigenvalue Projector, and Eigenspace Scoring. Each part will provide a comprehensive explanation of the methodology and its implementation.

A. Task Statement

In this work, we specify the loss of a model as $\mathcal{L}(\mathcal{D}, \boldsymbol{f})$, where $\mathcal{D}$ and $\boldsymbol{f}$ are the input data and filter set, respectively. Then we formulate the pruning task as

$$\begin{aligned} &\arg\min_{\boldsymbol{f}^* \in \boldsymbol{f}} \mathcal{L}(\, jD, \boldsymbol{f}) - \mathcal{L}(\mathcal{D}, f^i = 0) \\ &\text{s.t. } \boldsymbol{f} = \left\{ f_1^1, f_1^2, \ldots, f_k^n \right\} \end{aligned} \tag{1}$$

where f^i implies that the ith filter is removed from a specified layer and the superscripts and subscripts of f represent the filter index and layer index, respectively. At the same time, the $\boldsymbol{f}^*$ is used to stand for the filter set after the removal of filters.

By observing (1), the core target of the pruning task is to keep the $\boldsymbol{f}^*$ as small as possible while recovering the accuracy of the slimmed model maximally. Motivated by the random filter removal strategy in [26], we remove filters randomly from each layer when a fined-score is specified. Simultaneously, the random strategy has been verified efficiently to prune networks in [26] when the importance of each convolutional layer is confirmed; therefore, the random filter removal by the importance of eigenspace is also convincing.

In particular, we define a pruning set $\mathbf{P} = \{P_1, P_1, \ldots, P_k\}$ to store fined-scores of each eigenspace, then these scores will be used to assign a pruning ratio to each convolutional layer. Therefore, the core task of LiAP is to identify the value of P for each eigenspace.

B. Eigenvalue Projector

Previous weight-based pruning methods are prone to being affected by the weight information from the parameter space. Therefore, we adopt the SVD in [20] to project the weight information of each convolutional layer from parameter space to eigenspace. The parameter space and eigenspace can be denoted as $\boldsymbol{\Omega}^{(p)} \in \mathbb{R}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}}$ and $\boldsymbol{\Omega}^{(e)} \in \mathbb{R}^{\mathcal{I}\times\mathcal{O}}$, respectively. Notably, the $\mathcal{I}$ and $\mathcal{O}$ are the number of input and output filters for a layer, and $\mathcal{H}$ and $\mathcal{W}$ are the size of height and width of a filter. Inspired by the projection between parameter space and eigenspace in [20], we adopt the "Matrix Compression" and "Matrix Decomposition" techniques to achieve information approximation, namely, using the eigenvalues in eigenspace to approximate the weight information in parameter space for each layer.

1) Matrix Compression: The weight information of each convolutional layer usually has four dimensions with $\mathcal{I} \times \mathcal{H} \times \mathcal{W} \times \mathcal{O}$, which is difficult to decompose directly by SVD. Therefore, dimension compression strategies are essential to simplify the projection process. Here, we adopt the average folding from [20] to squeeze the original weight tensor

$$\begin{aligned} \boldsymbol{H}^{\text{aver}} &= \frac{\sum_h \mathbf{M}[:, :, h, :]}{|\boldsymbol{H}|} \\ \mathbf{M}^{\text{aver}} &= \frac{\sum_w \boldsymbol{H}^{\text{aver}}[:, :, w]}{|\boldsymbol{W}|} \\ \text{s.t. } \boldsymbol{H}^{\text{aver}} &\in \boldsymbol{\Omega}^{(p)}, \boldsymbol{W} \in \boldsymbol{\Omega}^{(p)} \\ \mathbf{M} &\in \mathbb{R}^{\mathcal{I}\times\mathcal{O}\times\mathcal{H}\times\mathcal{W}} \\ \mathbf{M}^{\text{aver}} &\in \mathbb{R}^{\mathcal{I}\times\mathcal{O}} \end{aligned} \tag{2}$$

where $\mathbf{M}$ stands for the original weight tensor that has four dimensions, $\boldsymbol{H}$ and $\boldsymbol{W}$ are the weight tensors that only have a 1-D filter over height or width, respectively. The $|\cdot|$ operator will generate the size of a specified matrix, and the $\boldsymbol{H}^{\text{aver}}$ implies $\mathbf{M}$ is folded along the height dimension. After folding of $\mathbf{M}$ through filter height and width, $\mathbf{M}$ can be squeezed as $\mathbf{M}^{\text{aver}}$ that has only two dimensions, which is suitable to achieve weight information projection via SVD.

2) Weight Matrix Decomposition: The approximation between parameter space and eigenspace with less loss is the key to capturing the information of weight via eigenvalues, namely, we adopt the eigenvalues to reflect the importance of each convolutional layer.

Inspired by the study in [20], the SVD can be used to decompose the weight matrix into the product of a left singular matrix $\mathbf{L}$, an eigenvalue matrix $\mathbf{S}$, and a right singular matrix $\mathbf{R}$

$$\begin{aligned} \mathbf{LSRR}^{\mathrm{T}}\mathbf{S}^{\mathrm{T}}\mathbf{L}^{\mathrm{T}} &= \mathbf{M}^{\text{aver}}\mathbf{M}^{(\text{aver},\mathrm{T})} \\ \mathbf{R}^{\mathrm{T}}\mathbf{S}^{\mathrm{T}}\mathbf{L}^{\mathrm{T}}\mathbf{LSR} &= \mathbf{M}^{(\text{aver},\mathrm{T})}\mathbf{M}^{\text{aver}} \\ \text{s.t. } \mathbf{S} &= \mathbf{S}_1, \mathbf{S}_2, \ldots, \mathbf{S}_n \end{aligned} \tag{3}$$

where the superscript T stands for the transpose of the specified matrix. Due to the equivalence between the original matrix and decomposed matrices, the eigenvalues in $\mathbf{S}$ can be used to capture the weight information from each convolutional layer, i.e., the number of eigenvalues can reflect the valid dimension of the original parameters, and the absolute value of each eigenvalue responds to the strength of the projection along the specified dimension.

Motivated by the attributes of decomposed eigenvalues, we can use the distribution of eigenvalues within each eigenspace to measure the importance of each convolutional layer and then yield a fine score to induce the pruning of CNNs. That is, if the distribution of eigenvalues is flat, the corresponding eigenspace will be important, and fewer filters will be removed from the original convolutional layer. It can be explained that if the distribution of eigenvalues is flat, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

strength of eigenvalues along all dimensions is similar, and it is hard to determine which dimension is vital. Consequently, more filters should be maintained, and vice versa.

C. Eigenspace Scoring Via KL Divergence

1) Distribution Measurement: Based on the above analysis, the distribution of eigenvalues can reflect the importance of the eigenspace by a fined-score, and then obtain the pruning ratio of each convolutional layer. Consequently, we adopt the KL divergence to measure the distance among eigenvalue distributions, as well as derive the pruning ratio of each layer by scoring distance measurement.

2) KL Divergence: The KL divergence can measure the distance between two distributions and then evaluate the information variance within distributions. Specifically, when a distribution is specified, the KL divergence can be used to calculate how far another distribution is away from the specified one, and quantify the variance of distributions with a fined-score. The KL divergence can be described by the discrete distributions

$$\mathbb{D}_{\mathrm{KL}}(Q_1||Q_2) = \sum_i Q_1(i) \ln\left(\frac{Q_1(i)}{Q_2(i)}\right) \tag{4}$$

where $Q(i)$ stands for a distribution that is parameterized by discrete variables i. In our work, the variable i can be represented by the eigenvalues S, and then we write distributions of eigenvalues as $\boldsymbol{d} = \{d_1(\mathbf{S}), d_2(\mathbf{S}), \ldots, d_k(\mathbf{S})\}$, where $d(\cdot)$ stands for the distribution of eigenvalues.

To build a universal comparison among distributions over all eigenspaces, a base distribution is selected from the eigenspaces to provide the reference comparison. That is, other distributions will be compared with the base distribution via KL divergence to yield different fined-scores with the same magnitude. In this article, the base distribution of eigenvalues among all eigenspaces can be screened by the variances

$$d^{(b)} = \{d|d(\mathbf{S}), \max(v(\mathbf{S}))\} \tag{5}$$

where $d^{(b)}$ stands for the base distribution and $v(\mathbf{S})$ will calculate the variance of the distribution for eigenvalues. When the variance is large, the shape of the corresponding distribution of eigenvalues is flat, which implies the eigenspace is important. Simultaneously, $\max(\cdot)$ can yield the max element over enclosed values to pick up the base distribution. Then we can calculate the KL divergence of each distribution according to the $d^{(b)}$

$$\mathrm{KL}_i = \sum_S d_i(\mathbf{S}) \ln \frac{d_i(\mathbf{S})}{d^{(b)}(\mathbf{S})}. \tag{6}$$

The value of KL will reflect how far each distribution is away from $d^{(b)}$. A larger KL divergence value indicates a greater dissimilarity between the corresponding distribution and $d^{(b)}$, suggesting that such distributions should be assigned lower weights or importance. To avoid the "0" values of KL, we relocate the KL to nonzero space through the exponential function

$$\mathrm{KL}_i^{(e)} = e^{\mathrm{KL}_i}. \tag{7}$$

As a result, a series of $\mathrm{KL}_i^{(e)}$ can be used to guide the removal of filters. In Section III-C3, we will introduce a novel scoring mechanism to prune each convolutional layer by $\mathrm{KL}_i^{(e)}$.

3) Scoring Each Eigenspace: For assigning the exact pruning ratio to each convolutional layer, we quantify the $\mathrm{KL}^{(e)}$ as specific fined-scores to determine how the percentage of filters will be removed from a convolutional layer. Specifically, we utilize the softmax to transfer values $\mathrm{KL}^{(e)}$ between 0 and 1 with the summation of 1, which is suitable to represent the pruning ratio of each layer

$$\mathrm{KL}_i^{(\mathrm{soft})} = \frac{\mathrm{KL}_i^{(e)}}{\sum_{i=1}^{n} \mathrm{KL}_i^{(e)}} \tag{8}$$

where $\mathrm{KL}^{(\mathrm{soft})}$ can be used to instruct the pruning of each layer, which satisfies $\mathrm{KL}_i^{(\mathrm{soft})} = P_i$. Correspondingly, we can confirm the pruning ratio of each layer quickly when an overall ratio $p^{(o)}$ is specified

$$P_i^{(o)} = P_i * p^{(o)}. \tag{9}$$

Then we can obtain the final pruning set $\mathbf{P}^{(o)} = \{P_1^{(o)}, P_2^{(o)}, \ldots, P_k^{(o)}\}$ that contain exact pruning ratio for each convolutional.

Reviewing the analysis of LiAP in the preceding parts, we conclude the overflow of LiAP in Algorithm 1. Operators $\mathrm{SVD}(\cdot)$ and $\mathrm{softmax}(\cdot)$ stand for the decomposition and softmax, respectively. After we derive the pruning ratio set $\boldsymbol{P}^{(o)}$, filters can be removed proportionally from the original convolutional layer, and then the slimmed model will be retrained carefully to recover the baseline.

IV. Experimental Study

In this section, we present a series of experiments conducted on VGG-16 [27] and ResNet-50 [28] using four remote sensing datasets: AID [29], PatternNet [30], NWPU-RESISC45 [4], and WHU-RS19 [31] for scene classification tasks. As illustrated in Fig. 2, these datasets differ from traditional optical datasets like CIFAR and ImageNet in that they require manual partitioning into training and testing sets. To comprehensively evaluate the effectiveness of LiAP, we adopt two partitioning ratios: 8:2 and 5:5. Additionally, we introduce a novel metric "Gdays" to quantify the computational complexity of various pruning methods by measuring the time cost on a specified GPU. This metric provides a practical assessment of the efficiency and scalability of the proposed approach.

A. Experimental Setting

1) *VGG-16 [27], ResNet-50 [28] (Fully Trained):*
   a) Training epochs are set as 150 for AID [29], NWPU-RESISC45 [4], and WHU-RS19 [31], as well as 50 for PatternNet [30].
2) *VGG-16 [27], ResNet [28] (Undertrained):*
   a) Training epochs are set as 50 for AID [29], NWPU-RESISC45 [4], and WHU-RS19 [31], as well as 20 for PatternNet [30].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

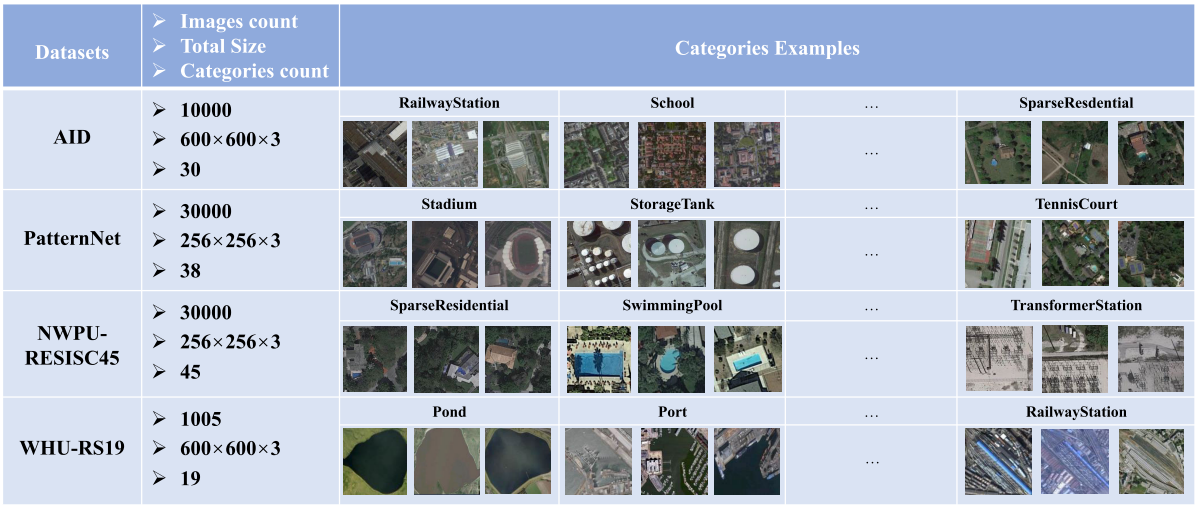


Fig. 2. Visualization of AID, NWPU-RESISC45, PatternNet, and WHU-RS19.

Algorithm 1 LiAP

Input: The input data: $\boldsymbol{D}$;
The weight tensor: $\mathbf{M}$;
The overall pruning ratio: $p^{(o)}$;
The SVD operator: SVD($\cdot$);
The softmax operator: softmax($\cdot$).
Output: The pruning ratio set for all layers: $\mathbf{P}^{(o)}$.
Squeeze the $\mathbf{M}$ to $\mathbf{M}^{aver}$ by Equation (2):

$$\mathbf{M}^{aver} \leftarrow \mathbf{M};$$

Obtain eigenvalues of each layer by Equation (3):

$$\boldsymbol{S} \leftarrow \text{SVD}(\mathbf{M}^{\text{aver}}).$$

Represent the distribution of eigenvalues $d(\mathbf{S})$ for each convolutional layer.
Calculate the KL divergence of eigenvalues in terms of Equation (4):

$$KL \leftarrow d(\mathbf{S}).$$

Select the base distribution $d^{(b)}(\mathbf{S})$ by the variance $v(\mathbf{S})$ according to Equation (5):

$$d^{(b)}(\mathbf{S}) \leftarrow v(\mathbf{S}).$$

Induce KL as $KL^{(e)}$ via Equation (7) to avoid the zero-values trap and then enter into nonzero space.
Quantify each eigenspace with a fined-score $KL^{(soft)}$ via softmax:

$$KL^{(soft)} \leftarrow softmax(KL^{(e)})$$

Yield $\mathbf{P}^{(o)}$ through $KL^{(soft)} * p^{(o)}$.

3) *Common Setting:*
   a) *Optimizer:* SGD.
   b) *Learning Ratio:* 0.01, 0.001, 0.0005, 0.0001, and 0.00001 for every 30 training epochs.
   c) *Batchsize:* 64.
   d) *Momentum:* 0.9.
   e) *Training Epochs After Slimming:* 150 for AID [29], NWPU-RESISC45 [4], and WHU-RS19 [31], as well as 50 for PatternNet [30].
   f) *Learning Ratio After Slimming:* 0.00001.
   g) *Data Partition:* 8:2 and 5:5.

The baselines of the above four remote sensing scene datasets are listed in Table I. In this article, we crop each image size to 224 × 224, and therefore, the FLOPs and parameters are the same when the model is the same. In the following section, we conduct different experiments in terms of LiAP to observe the variation of accuracy, FLOPs, and parameters for different models and datasets.

Before we prune VGG-16 [27] and ResNet-50 [28], we calculate the score of each layer by (8), and then plot the specific score in Figs. 3 and 4, from which we can see the score of VGG-16 [27] for all different datasets is distinct, no matter whether the original model is fully trained. On the contrary, ResNet-50 [28] shows relatively higher similarity across all datasets, which can yield almost similar scores for each layer on different datasets. In terms of the scoring mechanism, we conduct ablation and comparison pruning experiments in the following section to verify the efficiency of LiAP.

B. Ablation Analysis

For VGG-16 [27] and ResNet-50 [28], we set the pruning ratio as 0.5 and 0.55 to remove filters from each convolutional layer in terms of LiAP. Especially, both the fully trained model (150/50 epochs) and undertrained model (50/20 epochs) are pruned to verify the efficacy of LiAP on AID [29], PatternNet [30], NWPU-RESISC45 [4], and WHU-RS19 [31], respectively.

1) Experiment on VGG-16 [27]: Results of VGG-16 [27] are shown in Tables II and III, which show the accuracy, FLOPs, and parameters after pruning with a ratio of 0.5 and 0.55, respectively.

When the data partition is set to 8:2, the accuracy of the pruned model on the AID [29] and PatternNet [30] datasets reaches the baseline performance at a pruning ratio of 0.55. Notably, the pruned model even surpasses the baseline model, achieving a 2% improvement in accuracy. However, a slight accuracy drop of 1% is observed for the NWPU-RESISC45 [4] and WHU-RS19 [31] datasets. In terms of computational efficiency, the FLOPs are reduced by approximately 80%

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I

BASELINE OF VGG-16 [27] AND RESNET-50 [28] ON DIFFERENT DATASETS WHEN MODELS ARE TRAINED WITH 150/50 EPOCHS (FULLY TRAINED) AND 50/20 EPOCHS (UNDERTRAINED)

Models	Datasets	Datasets-Ratio	Param (M)	FLOPs (G)	Acc-150/50 (%)	Acc-50/20 (%)
VGG-16 [27]	AID [29]	8:2	17.25	7.71	93.25	92.95
	PatternNet [30]		17.25	7.71	97.78 (50)	95.65 (20)
	NWPU-RESISC45 [4]		17.25	7.71	92.65	91.62
	WHU-RS19 [31]		17.24	7.71	98.08	93.75
	AID [29]	5:5	17.25	7.71	88.05	78.07
	PatternNet [30]		17.25	7.71	99.60 (50)	98.74 (20)
	NWPU-RESISC45 [4]		17.25	7.71	89.88	89.11
	WHU-RS19 [31]		17.24	7.71	76.04	50.35
ResNet-50 [28]	AID [29]	8:2	25.78	2.06	94.50	92.00
	PatternNet [30]		25.78	2.06	99.54 (50)	96.02 (20)
	NWPU-RESISC45 [4]		25.78	2.06	91.84	91.57
	WHU-RS19 [31]		25.77	2.06	98.08	97.12
	AID [29]	5:5	25.78	2.06	95.23	89.15
	PatternNet [30]		25.78	2.06	99.66 (50)	99.60 (20)
	NWPU-RESISC45 [4]		25.78	2.06	91.75	89.91
	WHU-RS19 [31]		25.77	2.06	1008	94.35

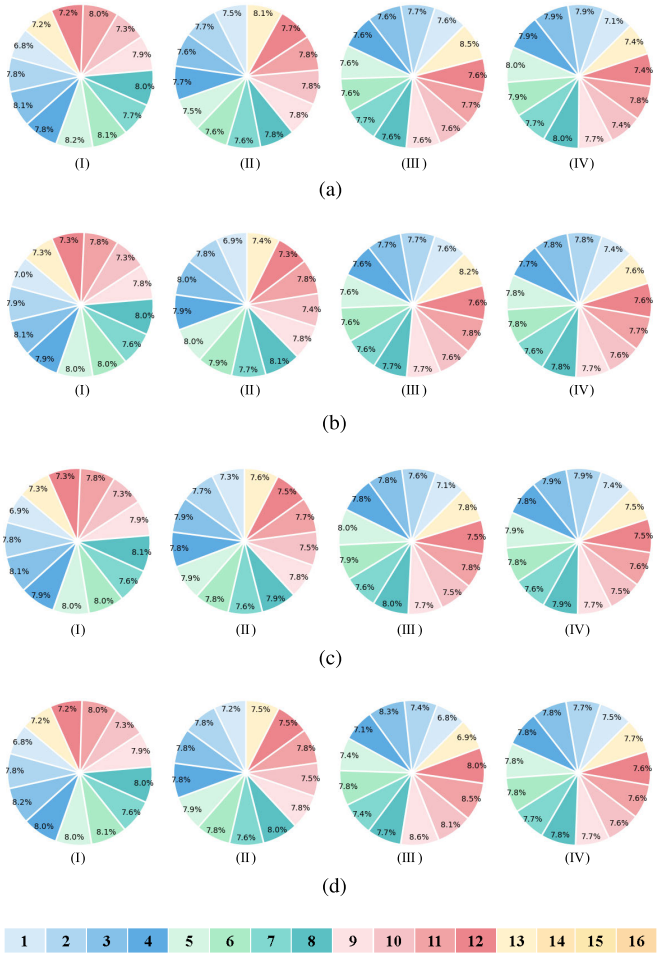


Fig. 3. Score of each convolutional layer for VGG-16 with different training epochs and data partition. The number around the Pie chart stands for the index of convolutional layers. (a) Score of undertrained VGG-16 when the data partition is 8:2. (b) Score of fully trained VGG-16 when the data partition is 8:2. (c) Score of undertrained VGG-16 when the data partition is 5:5. (d) Score of fully trained VGG-16 when the data partition is 5:5. (I) AID. (II) NWPU-RESISC45. (III) PatternNet. (IV) WHU-RS19.

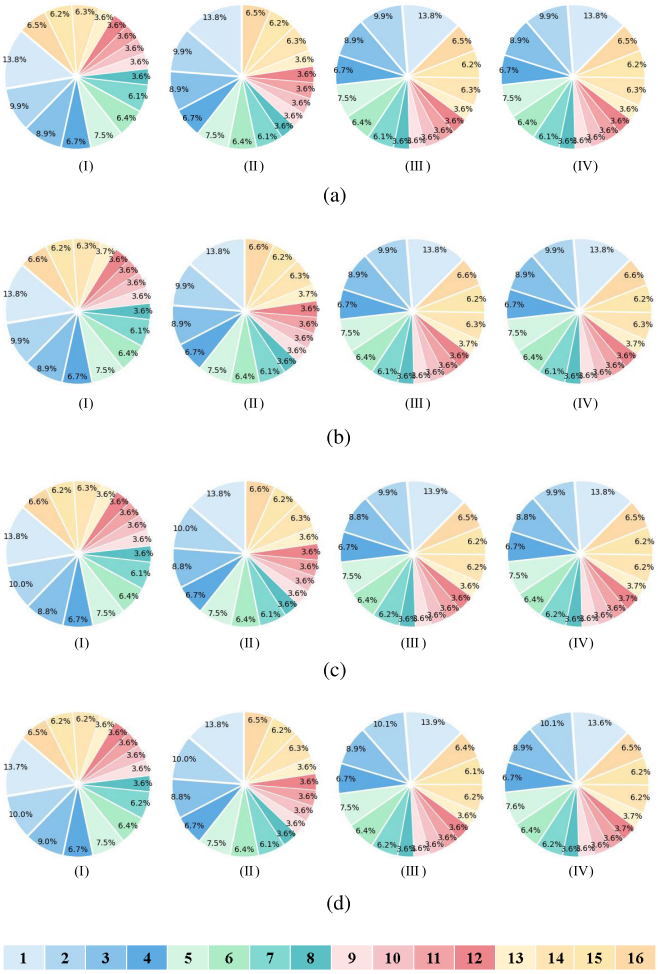


Fig. 4. Score of each convolutional layer for ResNet-50 [28] with different training epochs and data partition. The number around the Pie chart stands for the index of convolutional layers. (a) Score of undertrained ResNet-50 when the data partition is 8:2. (b) Score of fully trained ResNet-50 when the data partition is 8:2. (c) Score of undertrained ResNet-50 when the data partition is 5:5. (d) Score of fully trained ResNet-50 when the data partition is 5:5. (I) AID. (II) NWPU-RESISC45. (III) PatternNet. (IV) WHU-RS19.

across all datasets when VGG-16 [27] is pruned with a ratio of 0.5. Increasing the pruning ratio to 0.55 further reduces the FLOPs by nearly 81%. Interestingly, the reduction in FLOPs remains consistent regardless of whether the original model is fully trained or undertrained. Similarly, the number of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

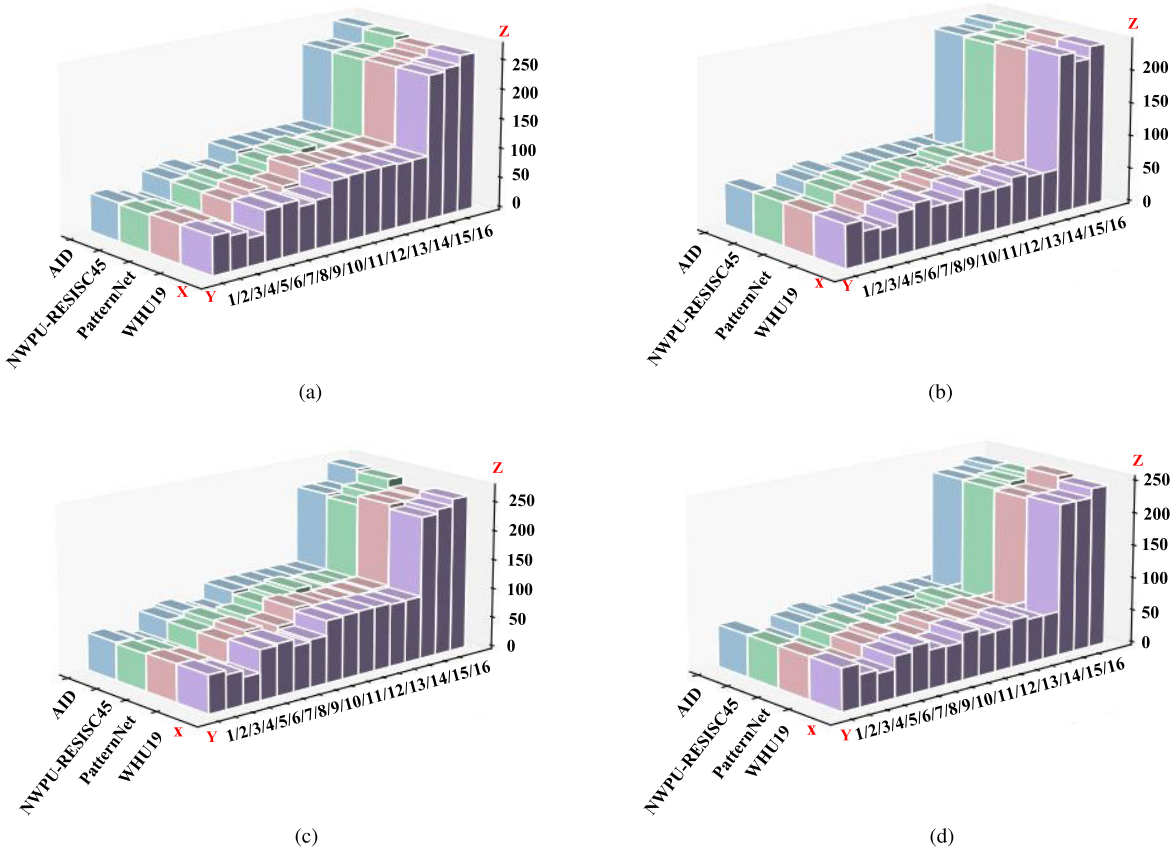


Fig. 5. Structure of ResNet-50 after pruning on AID, PatternNet, NWPU-RESISC45, and WHU-RS19 with different pruning ratio. The coordinate axis "*X*" represents four remote sensing datasets; The coordinate axis "*Y*" adopts the index of convolutional layers; The coordinate axis "*Z*" represents the number of filters. (a) Structure of fully trained ResNet-50 with 50% pruning ratio. (b) Structure of fully trained ResNet-50 with 55% pruning ratio. (c) Structure of undertrained ResNet-50 with 50% pruning ratio. (d) Structure of undertrained ResNet-50 with 55% pruning ratio.

parameters shows comparable reductions for both fully trained and undertrained models. Compared to the baseline, pruning with a ratio of 0.5 results in a 79% reduction in parameters across all datasets, while a pruning ratio of 0.55 removes 81% of the parameters. These experimental results demonstrate that LiAP is an effective approach for compressing VGG-16 [27] on the AID [29], PatternNet [30], NWPU-RESISC45 [4], and WHU-RS19 [31] datasets, even during the early stages of the original model's training.

When the data partition is adjusted to 5:5, the results exhibit a high degree of similarity with those obtained under the 8:2 partition. Specifically, the FLOPs and parameters remain stable at approximately 1.5 G and 3.6 M, respectively, for both undertrained and fully trained models when the pruning ratio is set to 0.5. Increasing the pruning ratio to 0.55 leads to further reductions, with FLOPs and parameters decreasing to around 1.3G and 3.2 M, which are more pronounced compared to the results under the 8:2 partition. In terms of accuracy, all pruned models consistently achieve performance close to the baseline, regardless of whether the original model is fully trained. This demonstrates the robustness and effectiveness of the pruning approach across different data partitioning scenarios.

2) Experiment on ResNet-50 [28]: Results on ResNet-50 [28] are shown in Tables II and III, which include accuracy, FLOPs, and parameters, respectively.

When the data partition is 8:2, the pruning ratio is set as 0.5 and 0.55 to test the performance of the slimmed model with LiAP. Compared with VGG-16 [27], ResNet-50 [28] exhibits more similarity among the listed metrics for all datasets. Surprisingly, the slimmed ResNet-50 [28] can outperform the baseline model in accuracy for all datasets. Specifically, the slimmed model can achieve almost 96%, 99%, 95%, and 99% for AID [29], PatternNet [30], NWPU-RESISC45 [4], and WHU-RS19 [31], respectively.

With respect to FLOPs and parameters, slimmed ResNet-50 [28] demonstrates higher similarities and coherence, which achieves 30% and 36% reductions in FLOPs when the original model is pruned by 0.5 and 0.55, as well as achieves 38% and 52% reductions in parameters when the pruning ratio is 0.55 for all datasets. Similar to the observations with VGG-16 [27], the reductions in FLOPs and parameters remain consistent regardless of whether the original ResNet-50 [28] is fully trained or undertrained. This further validates that LiAP is capable of effectively pruning ResNet-50 [28] even during the early stages of training.

When the data partition is adjusted to 5:5, the reductions in FLOPs and parameters remain nearly identical to those observed under the 8:2 partition. Notably, the accuracy of all slimmed models also remains consistent, regardless of whether the original models are undertrained or fully trained. This further underscores the robustness and stability of the pruning approach across different data partitioning scenarios.

The comprehensive experimental results presented above unequivocally validate the efficacy of LiAP for both VGG-16

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE II

50% PRUNING OF VGG-16 [27] AND RESNET-50 [28] ON DIFFERENT DATASETS WHEN MODELS ARE TRAINED WITH 150/50 EPOCHS (FULLY TRAINED) AND 50/20 EPOCHS (UNDERTRAINED)

Models	Datasets	Datasets-Ratio	Param (M)	FLOPs (G)	Acc-150/50 (%)	Acc-50/20 (%)
VGG-16 [27]	AID [29]	8:2	3.66	1.51	94.05	93.45
	PatternNet [30]		3.66	1.53	99.69 (50)	99.56 (20)
	NWPU-RESISC45 [4]		3.46	1.49	89.87	86.79
	WHU-RS19 [31]		3.64	1.56	96.63	95.67
	AID [29]	5:5	3.66	1.50	90.40	89.40
	PatternNet [30]		3.65	1.54	99.50 (50)	98.31 (20)
	NWPU-RESISC45 [4]		3.61	1.61	88.28	86.92
	WHU-RS19 [31]		3.64	1.54	84.10	85.94
ResNet-50 [28]	AID [29]	8:2	15.98	1.33	96.15	96.10
	PatternNet [30]		15.98	1.33	99.80 (50)	99.79 (20)
	NWPU-RESISC45 [4]		15.99	1.33	95.24	95.11
	WHU-RS19 [31]		15.99	1.33	99.52	99.04
	AID [29]	5:5	15.98	1.33	93.44	94.49
	PatternNet [30]		15.99	1.33	99.61 (50)	99.56 (20)
	NWPU-RESISC45 [4]		15.98	1.33	94.10	93.50
	WHU-RS19 [31]		15.96	1.33	100	99.77

TABLE III

55% PRUNING OF VGG-16 [27] AND RESNET-50 [28] ON DIFFERENT DATASETS WHEN MODELS ARE TRAINED WITH 150/50 EPOCHS (FULLY TRAINED) AND 50/20 EPOCHS (UNDERTRAINED)

Models	Datasets	Datasets-Ratio	Param (M)	FLOPs (G)	Acc-150/50 (%)	Acc-50/20 (%)
VGG-16 [27]	AID [29]	8:2	3.66	1.51	93.55	93.35
	PatternNet [30]		3.66	1.53	99.62 (50)	99.57 (20)
	NWPU-RESISC45 [4]		3.45	1.52	88.97	85.63
	WHU-RS19 [31]		3.62	1.54	97.12	96.63
	AID [29]	5:5	3.28	1.45	89.50	89.50
	PatternNet [30]		3.23	1.50	99.25 (50)	99.12 (20)
	NWPU-RESISC45 [4]		3.22	1.50	85.94	85.60
	WHU-RS19 [31]		3.26	1.48	83.56	90.32
ResNet-50 [28]	AID [29]	8:2	15.98	1.33	96.25	95.90
	PatternNet [30]		15.98	1.33	99.79(50)	99.62 (20)
	NWPU-RESISC45 [4]		15.98	1.32	95.21	95.03
	WHU-RS19 [31]		15.99	1.32	99.04	98.56
	AID [29]	5:5	15.98	1.21	93.88	94.27
	PatternNet [30]		15.96	1.21	99.56 (50)	99.51 (20)
	NWPU-RESISC45 [4]		15.96	1.22	93.37	93.23
	WHU-RS19 [31]		15.98	1.22	100	99.87

and ResNet-50 across various remote sensing scene datasets, demonstrating its robustness under different data partitioning scenarios. Given the superior accuracy performance of the slimmed ResNet-50 [28], we adopt it as the backbone model to compare the representation ability of the pruned model with several SOTA NAS-based and pruning-based methods in Section IV-C.

C. Comparison Study

Given that the 8:2 data partition yields higher accuracy, we conduct our comparative experiments using this partition ratio. In this section, we evaluate LiAP against several SOTA methods, including NAS-based approaches such as "Random Search" [15], "Gradient-based" [16], "SceneNet" [17], and "RSNet" [18], as well as pruning-based techniques like "Taylor Pruning" [43], "Network Slimming" [43], "EFPF" [20], and "DCMOP" [51]. In addition to accuracy, FLOPs, and parameter counts, we introduce searching cost as a metric to assess the efficiency of these methods. Specifically, a lower GDays value indicates that the corresponding method is more practical for real-world deployment.

The results are shown in Table IV. On the AID [29] dataset, our method achieves slightly lower accuracy compared to "EFPF" [20] and "DCMOP" [51], but remains competitive with other approaches. Notably, our method reduces FLOPs to 1.32G, achieving the best performance among all pruning-based methods, although it falls short of the random search method. In terms of parameter reduction, our approach outperforms most pruning-based methods with the exception of "DCMOP" [51]. Additionally, our method demonstrates superior efficiency, requiring a searching cost of less than 3 h, which is significantly lower than both NAS-based methods and "DCMOP" [51].

These two categories of model compression techniques are also evaluated on the NWPU-RESISC45 [4] dataset, which includes the additional NAS-based methods "SceneNet" [17] and "RSNet" [18]. Among these, "SceneNet" [17], "RSNet" [18], and "Random Search" [15] achieve the best performance in parameter compression, searching cost, and FLOPs reduction, respectively. However, NAS-based methods exhibit significant resource consumption due to their substantially higher searching costs, which are orders of magnitude greater

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF DIFFERENT METHODS WITH OURS (LIAP). THE "GDAYS" MEANS THE TIME COSTS OF CORRESPONDING METHODS WITH ONE 2080TI

Dataset	Method		Top1-Accuracy	FLOPs	Parameters	Searching Cost (GDays)
AID [29]		Baseline	94.50%	2.06	25.78M	-
	NAS-Based	Random Search [15]	96.10%	0.56G	5.10M	9.6
		Gradient-Based [16]	91.12%	-	3.80M	-
	Pruning-Based	Taylor Pruning [43]	96.15%	1.66G	18.13M	0.13
		Network Slimming [44]	96.15%	1.81G	18.02M	0.15
		EFPF(α=0.0020) [20]	96.90%	1.87G	17.98M	0.13
		DCMOP [51]	97.00%	1.79G	15.49M	1.8
		Ours	**96.25%**	**1.32G**	**15.99M**	**0.13**
PatternNet [30]		Baseline	99.54%	2.06	25.78M	-
	Pruning-Based	Taylor Pruning [43]	99.80%	1.47G	18.33M	0.12
		Network Slimming [44]	99.85%	1.86G	17.42M	0.13
		EFPF(α=0.0020) [20]	99.47%	1.87G	17.99M	0.13
		DCMOP [51]	99.79%	1.79G	19.40M	2.5
		Ours	**99.79%**	**1.32G**	**15.99M**	**0.13**
NWPU-RESISC45 [4]		Baseline	91.84%	2.06	25.78M	-
	NAS-Based	SceneNet [17]	95.22%	9.47G	1.02M	10
		RSNet [18]	92.66%	1.28G	1.40M	2
		Random Search [15]	96.56%	0.56G	5.10M	9.6
	Pruning-Based	Taylor Pruning [43]	94.80%	1.59G	18.29M	0.13
		Network Slimming [44]	94.53%	1.93G	17.66M	0.15
		EFPF(α=0.0020) [20]	95.02%	1.87G	17.99M	0.13
		DCMOP [51]	95.48%	1.79G	20.71M	1.8
		Ours	**95.21%**	**1.32G**	**15.99M**	**0.13**
WHU-RS19 [31]		Baseline	98.08%	2.06	25.78M	-
	Pruning-Based	Taylor Pruning [43]	98.73%	1.71G	17.78M	0.05
		Network Slimming [44]	98.91%	1.88G	16.97M	0.05
		EFPF(α=0.0020) [20]	99.90%	1.87G	17.99M	0.05
		DCMOP [51]	99.99%	1.69G	19.40M	1.5
		Ours	**99.04%**	**1.32G**	**15.99M**	**0.13**

than those of pruning-based methods. In comparison to other pruning methods, our LiAP approach demonstrates superior performance in reducing both FLOPs and parameters. Additionally, it recovers the baseline performance within just 0.13 GPU days, highlighting its effectiveness for the NWPU-RESISC45 [4] dataset.

For the PatternNet [30] and WHU-RS19 [31] datasets, we focus exclusively on comparing pruning-based methods to evaluate the performance of slimmed models. Notably, all the listed pruning methods enhance accuracy post-pruning, indicating significant redundancy in the filters of ResNet-50 [28] for these datasets. Among these methods, our LiAP achieves the most substantial reductions in both parameters and FLOPs. Specifically, on PatternNet [30] and WHU-RS19 [31], our method improves FLOPs and parameter reductions by nearly 20% compared to "DCMOP" [51], while also requiring a lower searching cost. Consequently, our approach demonstrates SOTA performance in compressing CNNs for scene classification across the AID [29], PatternNet [30], NWPU-RESISC45 [4], and WHU-RS19 [31] datasets.

D. Visualization of Model Structure

In this section, we visualize the structure of the slimmed ResNet-50 [28] under an 8:2 data partition to analyze the distribution of filters across convolutional layers. Both undertrained and fully trained models are illustrated in Fig. 5.

For the fully trained ResNet-50 [28], the structures of the slimmed models are depicted in Fig. 5(a) and (b) with pruning ratios of 0.5 and 0.55, respectively. We observe that filters in the shallow layers remain largely unchanged, while those in the deeper layers exhibit significant variation. Specifically, as the pruning ratio increases, filters in the deeper layers are more likely to be removed. This suggests that shallow layers hold relatively greater importance compared to deep layers. Additionally, the slimmed models demonstrate notable structural similarities across the four datasets, indicating that ResNet-50 [28] exhibits consistent feature representation capabilities when applied to similar datasets.

For the undertrained original model, the structures of the slimmed models are shown in Fig. 5(c) and (d). In comparison to the structures in Fig. 5(a) and (b), the undertrained models with the same pruning ratios display higher structural similarity, with only minor variations in the deep layers. This observation confirms that LiAP is robust to parameter updates in the original model, and it can achieve excellent pruning results even when the original model is undertrained.

V. CONCLUSION

This article proposes the LiAP to prune the prevalent CNN structure of VGG-16 and ResNet-50 on AID, PatternNet, NWPU-RESISC45, and WHU-RS19. Compared with previous pruning methods, LiAP can achieve reliable pruning results

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

for undertrained models by leveraging the eigenvalues of each weight matrix. Especially, LiAP can automatically assign each layer with a sparse ratio when an overall pruning ratio is specified, which can achieve end-to-end pruning without introducing extra hyperparameters. Additionally, LiAP can be deployed into the satellite expediently due to lower search costs. In future work, we will extend the LiAP from the CNN model to other backbones to achieve comprehensive compression.

REFERENCES

[1] G. Cheng, X. Xie, J. Han, L. Guo, and G.-S. Xia, "Remote sensing image scene classification meets deep learning: Challenges, methods, benchmarks, and opportunities," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3735–3756, 2020.

[2] P. Du, E. Li, J. Xia, A. Samat, and X. Bai, "Feature and model level fusion of pretrained CNN for remote sensing scene classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 8, pp. 2600–2611, Aug. 2019.

[3] G. Cheng, C. Ma, P. Zhou, X. Yao, and J. Han, "Scene classification of high resolution remote sensing images using convolutional neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, Jul. 2016, pp. 767–770.

[4] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[5] E. Li, J. Xia, P. Du, C. Lin, and A. Samat, "Integrating multilayer features of convolutional neural networks for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5653–5665, Oct. 2017.

[6] T. Liu et al., "Commonality feature representation learning for unsupervised multimodal change detection," *IEEE Trans. Image Process.*, vol. 34, pp. 1219–1233, 2025.

[7] J. Li, M. Gong, H. Liu, Y. Zhang, M. Zhang, and Y. Wu, "Multiform ensemble self-supervised learning for few-shot remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 4500416.

[8] T. Liu et al., "AEKAN: Exploring superpixel-based AutoEncoder Kolmogorov–Arnold network for unsupervised multimodal change detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5601114.

[9] L. Yang, S. Gu, C. Shen, X. Zhao, and Q. Hu, "Skeleton neural networks via low-rank guided filter pruning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 33, no. 12, pp. 7197–7211, Dec. 7211, 2023.

[10] H. Kang, "Accelerator-aware pruning for convolutional neural networks," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 30, no. 7, pp. 2093–2103, Jul. 2020.

[11] J. Guo, W. Zhang, W. Ouyang, and D. Xu, "Model compression using progressive channel pruning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 3, pp. 1114–1124, Mar. 2021.

[12] Y. Li et al., "Weight-dependent gates for network pruning," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 10, pp. 6941–6954, Oct. 2022.

[13] H. Zhang, L. Liu, H. Zhou, L. Si, H. Sun, and N. Zheng, "FCHP: Exploring the discriminative feature and feature correlation of feature maps for hierarchical DNN pruning and compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 10, pp. 6807–6820, Oct. 2022.

[14] Z. Wang, H. Xie, Y. Wang, H. Xu, and G. Jin, "DCFP: Distribution calibrated filter pruning for lightweight and accurate long-tail semantic segmentation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 7, pp. 6063–6076, Jul. 2024.

[15] C. Broni-Bediako, Y. Murata, L. H. B. Mormille, and M. Atsumi, "Searching for CNN architectures for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4701813.

[16] J. Chen et al., "Convolution neural network architecture learning for remote sensing scene classification," 2020, *arXiv:2001.09614*.

[17] A. Ma, Y. Wan, Y. Zhong, J. Wang, and L. Zhang, "SceneNet: Remote sensing scene classification deep learning network using multi-objective neural evolution architecture search," *ISPRS J. Photogramm. Remote Sens.*, vol. 172, pp. 171–188, Feb. 2021.

[18] J. Wang, Y. Zhong, Z. Zheng, A. Ma, and L. Zhang, "RSNet: The search for remote sensing deep neural networks in recognition tasks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2520–2534, Mar. 2021.

[19] J. Li et al., "Toward federated customized neural architecture search for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 4502012.

[20] Y. Lu, M. Gong, Z. Hu, W. Zhao, Z. Guan, and M. Zhang, "Energy-based CNN pruning for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 3000214.

[21] A. Bilal et al., "Improved support vector machine based on CNN-SVD for vision-threatening diabetic retinopathy detection and classification," *PLoS ONE*, vol. 19, no. 1, Jan. 2024, Art. no. e0295951.

[22] S. Tao, C. Shen, L. Zhu, and T. Dai, "SVD-CNN: A convolutional neural network model with orthogonal constraints based on SVD for context-aware citation recommendation," *Comput. Intell. Neurosci.*, vol. 2020, pp. 1–12, Oct. 2020.

[23] H. Liang, J. Cao, and X. Zhao, "Average descent rate singular value decomposition and two-dimensional residual neural network for fault diagnosis of rotating machinery," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–16, 2022.

[24] T. Peken, S. Adiga, R. Tandon, and T. Bose, "Deep learning for SVD and hybrid beamforming," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6621–6642, Oct. 2020.

[25] M. M. Bejani and M. Ghatee, "Theory of adaptive SVD regularization for deep neural networks," *Neural Netw.*, vol. 128, pp. 33–46, Aug. 2020.

[26] Y. Lu, Z. Guan, Y. Yang, W. Zhao, M. Gong, and X. Cai, "Entropy induced pruning framework for convolutional neural networks," in *Proc. AAAI Conf. Artif. Intell.*, Mar. 2024, vol. 38, no. 4, pp. 3918–3926.

[27] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1155–1167, Feb. 2019.

[28] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[29] G.-S. Xia et al., "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.

[30] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 197–209, Nov. 2018.

[31] D. Dai and W. Yang, "Satellite image classification via two-layer sparse coding with biased image representation," *IEEE Geosci. Remote Sens. Lett.*, vol. 8, no. 1, pp. 173–176, Jan. 2011.

[32] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2016, pp. 1–21.

[33] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.

[34] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Aging evolution for image classier architecture search," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2019, pp. 1–14.

[35] W. Jing, Q. Ren, J. Zhou, and H. Song, "AutoRSISC: Automatic design of neural architecture for remote sensing image scene classification," *Pattern Recognit. Lett.*, vol. 140, pp. 186–192, Dec. 2020.

[36] C. Peng, Y. Li, L. Jiao, and R. Shang, "Efficient convolutional neural architecture search for remote sensing image scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 6092–6105, Jul. 2021.

[37] J. Chen et al., "Contextual information-preserved architecture learning for remote-sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5602614.

[38] Z. Zhang, S. Liu, Y. Zhang, and W. Chen, "RS-DARTS: A convolutional neural architecture search for remote sensing image scene classification," *Remote Sens.*, vol. 14, no. 1, p. 141, Dec. 2021.

[39] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal brain damage," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 1990, pp. 598–605.

[40] B. Hassibi and D. G. Stork, "Second order derivatives for network pruning: Optimal brain surgeon," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 5, Nov. 1992, pp. 164–171.

[41] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Represent.*, Jan. 2018, pp. 1–42.

[42] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2016, pp. 1–12.

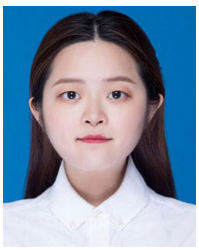
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.

[43] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," 2016, *arXiv:1611.06440*.
[44] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 2755–2763.
[45] X. Zhang, W. Xie, Y. Li, K. Jiang, and L. Fang, "REAF: Remembering enhancement and entropy-based asymptotic forgetting for filter pruning," *IEEE Trans. Image Process.*, vol. 32, pp. 3912–3923, 2023.
[46] J. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 5068–5076.
[47] R. Yu et al., "NISP: Pruning networks using neuron importance score propagation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 9194–9203.
[48] Z. Hu et al., "Scale-aware pruning framework for remote sensing object detection via multifeature representation," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025, Art. no. 5614215.
[49] H. Hu, R. Peng, Y.-W. Tai, and C.-K. Tang, "Network trimming: A data-driven neuron pruning approach towards efficient deep architectures," 2016, *arXiv:1607.03250*.
[50] Z. Hu, M. Gong, Z. Dong, Y. Lu, J. Li, and Y. Zhao, "Heterogeneity-aware pruning framework for personalized federated learning in remote sensing scene classification," *Knowl.-Based Syst.*, vol. 311, Feb. 2025, Art. no. 113007.
[51] Z. Hu, M. Gong, Y. Lu, J. Li, Y. Zhao, and M. Zhang, "Data customization-based multiobjective optimization pruning framework for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 4508514.



Yiheng Lu received the B.S. degree from Shantou University, Shantou, China, in 2018, and the Ph.D. degree from Xidian University, Xi'an, China, in 2024.

He holds a post-doctoral position at Xidian University. His research interests include deep learning and structure optimization.



Zhuping Hu is currently pursuing the Ph.D. degree with Xidian University, Xi'an, China.

Her research interests include deep learning, multiobjective evolution, and network structure optimization.



Wei Zhao (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees from Xidian University, Xi'an, China, in 2002, 2005, and 2015, respectively.

He is currently an Associate Professor with the School of Computer Science and Technology, Xidian University. His research direction is pattern recognition and intelligent systems, with specific interests in attributed graph mining and search, machine learning, signal processing, and precision guiding technology.



Ziyu Guan (Senior Member, IEEE) received the B.S. and Ph.D. degrees in computer science from Zhejiang University, Hangzhou, China, in 2004 and 2010, respectively.

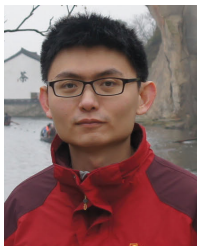
He worked as a Research Scientist with the University of California at Santa Barbara, Santa Barbara, CA, USA, from 2010 to 2012, and a Professor with the School of Information and Technology, Northwest University, Xi'an, China, from 2012 to 2018. He is currently a Professor with the School of Computer Science and Technology, Xidian University, Xi'an. His research interests include attributed graph mining and search, machine learning, expertise modeling and retrieval, and recommender systems.



Maoguo Gong (Fellow, IEEE) received the B.Eng. and Ph.D. degrees from Xidian University, Xi'an, China, in July 2003 and June 2009, respectively.

Since 2006, he has been a Teacher with Xidian University. He was promoted to an Associate Professor and a Full Professor in 2008 and 2010, respectively, with exceptive admission. He has published over 100 papers in journals and conferences and holds more than 20 granted patents as the first inventor. He is leading or has completed more than 20 projects as the Principle Investigator, funded by the National Natural Science Foundation of China and the National Key Research and Development Program of China. His research interests are broadly in the area of computational intelligence, with applications to optimization, learning, data mining, and image understanding.

Dr. Gong is the Executive Committee Member of Chinese Association for Artificial Intelligence and a Senior Member of Chinese Computer Federation. He was a recipient of the Prestigious National Program for Support of the Leading Innovative Talents from the Central Organization Department of China, the Leading Innovative Talent in Science and Technology from the Ministry of Science and Technology of China, the Excellent Young Scientist Foundation from the National Natural Science Foundation of China, the New Century Excellent Talent from the Ministry of Education of China, and the National Natural Science Award of China. He is an Associate Editor or an Editorial Board Member of over five journals, including IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.



Mingyang Zhang (Member, IEEE) received the B.Eng. degree in automation and the Ph.D. degree in pattern recognition and intelligent systems from Xidian University, Xi'an, China, in 2012 and 2018, respectively.

Since 2018, he has been a Lecturer with the School of Electronic Engineering, Xidian University. His research interests include computational intelligence and remote sensing image understanding.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 22:38:56 UTC from IEEE Xplore. Restrictions apply.